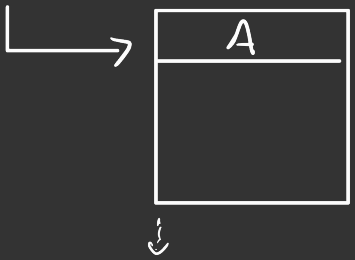
Singleton Design Pattern

Singleton Design Pattern

Singleton

→ Singleton class is a class for which only one object is created/instanciated. If we try to create another object it return the first object only.

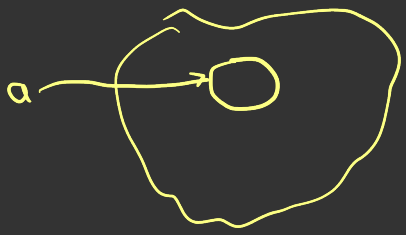
Impl^n :- Singleton



A a = new A();

Constructor is called when object created.

Heap → non-primitive data types are stored

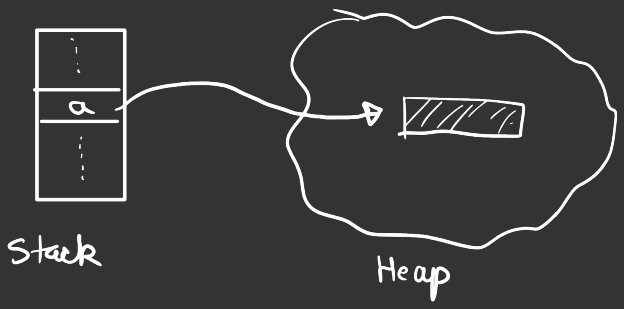


Stack ← Primitive data types are stored.

int, long ...

⊙ Under the Hood

```
A a = new A();
```



⊙ How to avoid creation of multiple Objects ?

⇒ Culprit is "new" Keyword. So jo bhi new Keyword dalke Constructor call karega, vo Object ban jayega.

Hence Constructor ko private kardo.

eg :-

```
class Singleton {
    Priate static Singleton instance;
    Private Singleton() {
        // Initialization.
    }
    Public static Singleton getInstance() {
        if (instance == null){
            instance = new Singleton();
        }
        return instance
    }
}
```

↳ The above solution is ok for single thread but it is not thread safe. In case of Multi-threadin problems will arise.

✳ Making it thread-safe

```
class Singleton {
    Priate static Singleton instance;
    Private Singleton() {
        // Initialization.
    }
    Public static Singleton getInstance() {
        if (instance == null){
            instance = new Singleton();
        }
        return instance
    }
}
```

→ critical section.

So, critical section pe lock lagana padega

```
class ThreadSafeLockingSingleton {
    Priate static ThreadSafeLockingSingleton instance;
    Private ThreadSafeLockingSingleton() {
        // Initialization.
    }
    Public Static ThreadSafeLockingSingleton getInstance() {
        synchronized (ThreadSafeLockingSingleton.class) {
            if (instance == null){
                instance = new Singleton();
            }
            return instance
        }
    }
}
```

lock for thread safety

* Locking and unlocking mechanism hai, kisi bhi threading environment me chahe vo koi bhi language ho, vo na expensive operation hota hai, bhot sari resource use hoti hai, aap hdd karke rakhte ho bhot sari threads ko. So hum chahte hai jitne kam time ke liye locking ho utna accha hai.

Yaha par jaise hi getInstance() method call hua hume lock lgga diya 😐. This can be improved.

Pehle check karo ki instance null hai ki nahi, agar null hai tab lock lago.

✳ Double Locking

```
class ThreadSafeLockingSingleton {
    Priate static ThreadSafeLockingSingleton instance;
    Private ThreadSafeLockingSingleton() {
        // Initialization.
    }
    Public Static ThreadSafeLockingSingleton getInstance() {
        if (instance == null){
            Synchronized (ThreadSafeLockingSingleton.class) {
                if (instance == null){
                    instance = new Singleton();
                }
            }
        }
        return instance
    }
}
```

We can avoid lock !

⇒ using Eager initialization.

eager initializati

```
public class ThreadSafeEagerSingleton {
    private static ThreadSafeEagerSingleton instance =
            new ThreadSafeEagerSingleton();

    public static ThreadSafeEagerSingleton getInstance() {
        // Initialization
    }

    public static ThreadSafeEagerSingleton getInstance() {
        return instance;
    }
}
```

Disadvantage :→ Only pratical in case of light weight object.

Conclusion

↳ ① Create a private Constructor

↳ ② Create a static instance [getInstance()] that returns the same instance everytime.

Practical Use Case

① Logging System.

② DB connection

③ Configuration Manager.